
Content Matters More Than Placement: Privileged Supervision in LLM Post-Training

Anonymous Author(s)

Affiliation

Address

email

Abstract

Privileged supervision gives a post-training teacher information unavailable at inference time, such as a verified solution or solver feedback. But does its benefit come from the information itself or from the channel through which it is delivered? We separate content from placement in neurosymbolic reasoning, where a language model generates formal programs checked by a theorem prover. Using Self-Distillation Policy Optimization (SDPO) on ProverQA and FOLIO, we vary both the supervision payload and its prompt slot. Content determines the outcome while placement has little effect: moving the same verified program between slots is a null, whereas replacing it with structured prover diagnostics reduces accuracy by 18 points, and an information-free context drives executable generation below 10 % — though the programs that still execute remain 92 % accurate, a collapse in formal-program generation rather than in logical accuracy. We further show that supervision quality can be screened before training using the teacher’s log-probability margin between verified and failed rollouts. The gate’s verdict tracks the policy rather than the string: the same verified context is unusable on the base model and strongly positive after a short scalar-reward warm start. Richer feedback is therefore not automatically better supervision: what the teacher is shown matters more than where it is shown.

1 Introduction

A neurosymbolic pipeline splits answering a logical question in two: a language model *translates* the problem into a formal program, and a symbolic engine *infers* an answer correct by construction with respect to it [Pan et al., 2023]. The burden therefore falls on translation: an 8B-class open model emits executable programs on only 52 %–59 % of FOLIO problems, and for Qwen2.5-7B the 62.0 %-versus-36.8 % gap between accuracy-on-executable and end-task accuracy is attributable to programs that never ran (Section C.4). Yet the verifier is no scalar oracle: Prover9 [McCune, 2005–2010] reports which premise failed and at which arity, and verifiable-reward RL keeps one number of it.

Self-distillation policy optimization (SDPO) [Hübotter et al., 2026] spends that information as *gradient* rather than as a prompt: a failed rollout is re-prompted with privileged context, and a successful re-prompted attempt is distilled into the un-prompted policy, which never sees the context at test time. The inference-time version does not work — a controlled study of solver-guided self-refinement on this pipeline reports no improvement over LOGIC-LM [Maheshwari and Sharma, 2026], consistent with models being unable to act on their own error signals [Huang et al., 2024]. But “privileged context helps” is two hypotheses in one coat: the teacher may succeed because of *what the context says*, or merely because it was *re-prompted through a distinguished channel*. Varying only the algorithm — SDPO versus GRPO [Shao et al., 2024] — confounds it with the information given.

37 **A payload $\times$ slot factorial** We hold base model, solver, reward, optimizer and budget fixed
 38 and vary only the privileged context (Section 3.3). One solver-verified program occupies both
 39 the demonstration slot (B-DEMO) and the feedback slot (D-SLOT), identifying placement with the
 40 payload byte-identical; that slot then carries the verified program, structured prover diagnostics
 41 (C-DIAG), or an information-free constant string (E-CONST), identifying content with the slot fixed.

42 Placement barely registers and content decides everything. Moving the identical program between
 43 slots moves ProverQA accuracy from 84.53 to 85.20 — 0.67 points, a null we report because the
 44 hypothesis was stated before the arms were run. Holding the slot fixed gives 85.20 for the verified
 45 program, 67.13 for diagnostics (−18.07) and 8.73 for the constant string. The two harmful payloads
 46 fail in different ways — distinct failure signatures, making the result more than a ranking. Diagnostics
 47 starve the objective of targets, since the loss is masked to prompts with a verified sibling and
 48 diagnostics rarely produce one: that arm discards 61.6 % of every batch against 17 % either side of it.
 49 The constant-string arm discards a healthy 17.5 % and collapses anyway, failing to parse on 1,216 of
 50 1,500 problems — a failure of executable-program generation, not of conditional logical accuracy
 51 (Section 5.3).

52 Verified and diagnostic context were separable before any gradient step. A *teacher-quality gate*
 53 (Section 3.4) scores a context by the change it induces in the teacher’s log-probability margin between
 54 a verified and a failed rollout. On 100 fixed prompts the verified-sibling channel scored +0.037 (74 %
 55 of pairs positive), the diagnostic channel negative (18 %). That score is a margin, not an accuracy;
 56 the corresponding cost after training is 40.7 accuracy points on the subset diagnostics were eligible
 57 to repair. We did not screen the constant-string channel, so its collapse is a result rather than a
 58 prediction.

59 What the gate measures is a property of the model–context pair, not of the feedback string in isolation.
 60 A verified program takes the warm-started policy’s next attempt from 70.2 % to 96.5 %, while the
 61 same instrument on the *base* model scores every context at or below plain prompting (Section 5.2).
 62 A scalar-reward warm start thus changes whether the model can exploit the same context: success
 63 depends not only on rich supervision being available, but on whether the current policy can consume
 64 it.

65 1.1 Contributions

- 66 1. **Content, not placement, governs privileged supervision.** A payload $\times$ slot factorial shows
 67 that moving a byte-identical verified program between slots is a null while changing what
 68 fills one slot spans the full range of outcomes, with the two harmful payloads failing through
 69 distinct mechanisms (Sections 3.3, 5.1 and 5.3).
- 70 2. **A pre-gradient gate for teacher context.** A log-probability margin test with a foreign-
 71 payload control and an echo audit separates a useful channel from a harmful one before
 72 training and orders their outcomes; its verdict depends on the model–context pair rather than
 73 on the feedback string alone (Sections 3.4 and 5.2).
- 74 3. **A reproducible verifier-as-reward stack.** We reproduce the reference LOGIC-LM Prover9
 75 evaluation and audit reward degeneracy, determinism and resource safety with a solver in
 76 the training loop (Section C.5, Appendix D).

77 2 Related Work

78 **Neurosymbolic translation to formal logic** The pipeline we study originates with LOGIC-LM
 79 [Pan et al., 2023], which prompts a language model to emit a solver-specific program and dispatches
 80 it to a symbolic backend — Prover9 [McCune, 2005–2010] for first-order logic. LINC [Olausson
 81 et al., 2023] majority-votes over multiple translations, trading sample cost for executability; SATLM
 82 [Ye et al., 2023] generalizes to declarative satisfiability. FOLIO [Han et al., 2024] supplies the
 83 human-annotated benchmark and ProverQA [Qi et al., 2025] a synthetic alternative whose labels are
 84 Prover9-generated and therefore solver-consistent by construction (Section 4.1). Across this line the
 85 translation model is trained, if at all, only by prompting: the solver consumes the model’s output
 86 rather than supervising it.

87 **Self-refinement and its limits** SELF-REFINE [Madaan et al., 2023] and REFLEXION [Shinn et al.,
 88 2023] established iterative, feedback-conditioned revision, and Huang et al. [2024] showed that when

the critic is the model itself, intrinsic self-correction does not improve reasoning and frequently degrades it. A theorem prover looks like the obvious escape — a sound external critic whose verdicts are not the model’s own opinion — but a controlled study of refinement, ensembling and chaining on LOGIC-LM [Maheshwari and Sharma, 2026] finds that gains come from *choosing among* candidate programs, not from *repairing* them, and execution-feedback-driven repair reports the same boundary [Chen et al., 2024, Olausson et al., 2024]. These results bound what in-context consumption can achieve; we test whether moving the same class of verifier information into post-training changes that conclusion, and find that the limiting factor is the content of the supervision rather than its timing alone.

Reinforcement learning from verifiable rewards RLVR post-trains policies against programmatically checkable outcomes, and GRPO [Shao et al., 2024] has become its default algorithm [DeepSeek-AI, 2025], with DAPO [Yu et al., 2025] addressing the degenerate case in which every rollout in a group scores identically and the advantage vanishes — the condition our viability gate measures. The signal throughout is scalar: whatever the verifier reports is compressed to one number before the policy sees it. Within formal reasoning, LogicPO [Viswanadha et al., 2025] finetunes an open model with DPO and KTO on a preference dataset for NL→FOL translation, and PRoSFI [Chen et al., 2026] applies process rewards over structured formal intermediates. Neither optimizes against a live solver in the loop, which is the axis this paper varies.

Self-distillation and rich feedback SDPO [Hübotter et al., 2026] is the mechanism we adopt: failed rollouts are re-prompted with privileged context, and a re-prompted rollout clearing a success threshold is distilled into the un-prompted policy. It fits because the context enters through a template rather than as a scalar, so an arbitrarily structured payload can be carried, and because the reference implementation ships a GRPO baseline differing only by the self-distillation block. We are precise about what our payload is: Prover9 emits no usable explanation of its own — the reference backend discards what it does produce — so C-DIAG carries a diagnostic layer we rebuilt (Section 3.1), a structured report of failure class, premise index and token position rather than solver prose. What SDPO does not establish, and what we add, is whether the *content* of that payload drives the gain or whether re-prompting on failure suffices by itself. Placement is a live hypothesis because interface changes move behaviour in this pipeline: a chat-template wrap collapses the student’s executability from 12.2 % to 0.9 % (Section 3.2). Distilling teacher distributions [Hinton et al., 2015, Agarwal et al., 2024] and self-training against a slowly moving target [Grill et al., 2020] share the mechanism but not the question.

Reward hacking in verifier-grounded settings Reward hacking is well characterized in general [Amodi et al., 2016, Skalse et al., 2022], and formal reasoning is usually treated as immune because the verifier is sound. Section 3.1 shows that soundness of the *prover* does not imply soundness of the *reward*: an inconsistent premise set is a valid input on which the prover behaves correctly and the reward is nevertheless maximal and meaningless.

3 Method

3.1 Task and reward

An instance is a triple (P, q, y^*) : a natural-language premise set, a question, and a gold label in $\{\text{True}, \text{False}, \text{Uncertain}\}$. A policy π_θ emits a program z in the LOGIC-LM surface syntax; a deterministic front end parses it, lowers it to Prover9 syntax, and invokes the prover, yielding $\mathcal{S}(z) = (\hat{y}, c, \phi)$ — a predicted label, a failure class, and a diagnostic string. A program that does not execute is scored *incorrect*, never routed to a fallback.

The teacher’s payload is this study’s independent variable, and for C-DIAG that payload is ϕ — which the reference implementation discards before any caller can see it. We rebuilt the layer: it attributes a failure to a premise index and token, separates lexer from parser errors and Prover9 exit codes from parse failures, and maps outcomes onto nine mutually exclusive classes (Appendix A).

Table 1: Experimental arms, one seed each (Section 6). Base model, prompt, solver, reward, optimizer and rollout budget are identical throughout; the *only* thing that varies is what the teacher sees and where it sees it. The *demo* slot continues the few-shot pattern and terminates at the program cue; the *feedback* slot appends an instruction-shaped block after that cue. GRPO is a scalar-reward reference rather than a controlled algorithmic contrast (Section 4.2).

Arm	Algorithm	Payload	Slot
A-GRPO	GRPO	— (scalar reward only)	—
B-DEMO	SDPO	verified correct program	demo
D-SLOT	SDPO	verified correct program	feedback
C-DIAG	SDPO	structured diagnostics	feedback
E-CONST	SDPO	constant: “The program failed.”	feedback
F-MIX	SDPO+GRPO	verified correct program	demo

$$R(z) = \begin{cases} 0.0 & \text{malformed, unparseable, or Prover9 errors,} \\ 0.1 & \text{executes, wrong answer,} \\ 1.0 & \text{executes, correct answer.} \end{cases} \quad (1)$$

137 The 0.1 tier gives a gradient toward executability without making executability sufficient. SDPO’s
138 success threshold is $\tau = 1.0$ (Appendix B).

139 Two kinds of executable program earn reward while encoding nothing: a *contradictory* premise set
140 entails every formula, and a *vacuous* one constrains nothing and returns `Unknown`, which maps
141 to “Uncertain”. We add a both-directions consistency check and a premise-side vacuity test — a
142 program is vacuous when every premise is a logical validity — and expose `contradictory_kb` and
143 `vacuous_kb` as failure classes tracked per training step and scored as failures. The more obvious
144 conclusion-groundedness test is unsound on ProverQA, whose `Uncertain` class is constructed using
145 the very pattern that test detects (Appendix D).

146 3.2 Self-distillation of solver feedback

147 We follow SDPO [Hübotter et al., 2026]. For a prompt $x = (P, q)$ the policy samples n rollouts.
148 Rollouts scoring below the success threshold are *re-prompted* with x , their own program z , and a
149 feedback string $f(z) = \text{TEMPLATE}(\phi(z))$, and sample a repaired program z' . When $R(z') \geq \tau$, the
150 distribution over z' is distilled into the un-prompted policy against a self-teacher updated at rate η ,
151 restricted to the top- k tokens. The feedback appears *only* inside post-training; at evaluation the policy
152 is queried without it, so any gain is a change in the policy rather than in the prompt.

153 **Why we expected the slot to matter** Before running the arms, wrapping the student’s LOGIC-LM
154 prompt in the chat template reduced executability from 12.2 % to 0.9 %, motivating our test of whether
155 teacher-context placement is similarly sensitive.

156 3.3 The payload $\times$ slot factorial

157 **B-DEMO vs. D-SLOT: placement, content fixed.** The payload is byte-identical — the same
158 solver-certified program for the same problem, from the same extraction path — and the arms differ
159 only in whether it continues the few-shot pattern or is written into an instruction-shaped block after
160 the pattern’s terminator. Any gap is attributable to the channel.

161 **D-SLOT vs. C-DIAG vs. E-CONST: content, placement fixed.** All three write into the feedback slot
162 and differ only in what it says: a verified correct program, structured diagnostics, or an information-
163 free constant. E-CONST is the null payload, so a deficit shared by all three would be a property of the
164 slot rather than of any content — which is what separates “the diagnostics are unhelpful” from “the
165 interface is unusable”, two claims a single feedback arm cannot distinguish.

166 **F-MIX.** F-MIX adds a $\lambda=0.5$ policy-gradient term so that groups without a verified sibling still
167 receive scalar-reward gradient instead of being dropped; details are in Appendix B.

168 3.4 Screening a channel before spending gradient steps

169 Every arm in Table 1 is a bet that the teacher can read its channel, and a bet that fails costs a full
170 post-training run to discover. We screen channels beforehand with a measurement that requires no
171 gradient steps.

172 Sample n rollouts per prompt from the policy under test and keep the prompts with at least one
173 verified-correct and one failed rollout. For such a pair (z^+, z^-) define the teacher’s *preference margin*
174 under context c as the difference in mean token log-probability,

$$m(c) = \frac{1}{|z^+|} \log \pi_{\text{teacher}}(z^+ | c) - \frac{1}{|z^-|} \log \pi_{\text{teacher}}(z^- | c), \quad (2)$$

175 and score a candidate channel by $g = m(c_{\text{channel}}) - m(c_{\text{plain}})$. A channel with $g \leq 0$ does not improve
176 the teacher’s ability to tell a correct program from a failed one. We correct g using a foreign-payload
177 control and reject measurements with excessive response–context n -gram overlap; both controls were
178 necessary in practice and are detailed in Appendix B. The gate costs one forward pass per response
179 per condition.

180 4 Experimental Setup

181 4.1 Data

182 We train on ProverQA [Qi et al., 2025] (4,966 instances) and report primary results on its 1,500-item
183 test split. The choice is driven by reward fidelity rather than size: FOLIO’s gold labels are human
184 annotations over natural language, so a *faithful* first-order logic translation is not guaranteed to
185 make the gold label derivable under Prover9, and that mislabeling noise would enter the reward
186 directly. ProverQA’s labels are Prover9-generated and therefore solver-consistent by construction.
187 We report FOLIO [Han et al., 2024] dev as out-of-distribution transfer, giving a “train synthetic, test
188 human-written” reading; the two sets are not pooled, and FOLIO is small ($n=204$), so we treat it as
189 directional OOD evidence.

190 The “Uncertain” prior shifts from 14.9 % in train to 35.9 % in test (Appendix C), so we report per-label
191 accuracy and macro-average alongside overall accuracy. This interacts directly with the vacuous-KB
192 channel of Section 3.1: unconstraining premises return **Unknown**, which is “Uncertain”, which is free
193 reward on more than a third of the test set. The test split is a clean 500/500/500 easy/medium/hard
194 partition and we report all three.

195 4.2 Models and post-training configuration

196 All runs use Qwen2.5-7B-Instruct [Team, 2025], train batch size 32, $n=8$ rollouts per prompt, learning
197 rate 1×10^{-6} , and 150 training steps; SDPO-specific settings follow the reference implementation
198 (Appendix B). Every trained arm and the viability gate below use this checkpoint; the untrained
199 calibration baselines of Appendix C.4 additionally cover Qwen2.5-7B base and Qwen2.5-3B base.

200 The upstream GRPO and SDPO configurations use mini-batch sizes 8 and 1 respectively; we therefore
201 treat GRPO as a scalar-reward reference rather than a controlled algorithmic contrast. All payload $\times$
202 placement comparisons use identical MB=1 optimization.

203 4.3 Metrics

204 We validate every 10 training steps, four samples per prompt, and track four quantities: **executability**
205 (*Exe*), the fraction of generations that parse and execute under Prover9; **end-task accuracy**, with
206 non-executable scored as incorrect; **failure composition** over the nine-class taxonomy of Section 3.1;
207 and **degeneracy rates**, contradictory- and vacuous-KB, per Section 3.1.

208 Non-executable programs are scored incorrect; we do not use LOGIC-LM’s chain-of-thought fallback,
209 because it would decouple the training reward from the reported metric and make executability
210 partially free. We likewise do not discard failing generations before aggregation.

Table 2: Main results. ProverQA test ($n=1,500$) is the primary metric; FOLIO dev ($n=204$) measures out-of-distribution transfer and is *not* pooled with it. Accuracy scores non-executable programs as incorrect throughout. All post-trained arms branch from the same warm-start checkpoint with a fresh optimizer, single seed. The payload and placement contrasts the paper claims share identical optimization; GRPO is a scalar-reward reference rather than a controlled algorithmic contrast (Section 4.2).

Method		ProverQA test							FOLIO	
		Overall	Easy	Medium	Hard	A	B	C	Macro	OOD
<i>No post-training</i>										
	Base model	36.27	56.80	37.00	15.00	30.21	29.12	48.05	35.80	32.84
<i>Post-trained — warm start</i>										
	GRPO warm start (step 30)	66.13	87.60	74.20	36.60	61.49	57.84	77.74	65.69	48.53
<i>Post-trained — scalar reward</i>										
A-GRPO	GRPO	84.67	95.80	92.60	65.60	81.91	81.47	89.98	84.45	46.08
<i>Post-trained — privileged context, demo slot</i>										
B-DEMO	SDPO, verified program	84.53	97.20	92.40	64.00	81.91	80.45	90.54	84.30	50.00
<i>Post-trained — privileged context, feedback slot</i>										
D-SLOT	SDPO, verified program	85.20	96.80	92.00	66.80	82.34	80.45	92.02	84.94	47.06
C-DIAG	SDPO, structured diagnostics	67.13	85.00	74.60	41.80	62.77	64.56	73.28	66.87	46.08
E-CONST	SDPO, constant string	8.73	8.40	9.40	8.40	8.30	8.96	8.91	8.72	32.84
<i>Post-trained — objective ablation</i>										
F-MIX	SDPO+GRPO, $\lambda=0.5$	84.73	94.80	88.80	70.60	80.00	80.04	93.14	84.39	40.20

4.4 Predicting post-trainability: the viability gate

Before training, we sample eight rollouts on 100 prompts and report (i) the fraction of groups with non-zero reward variance [Yu et al., 2025] and (ii) the fraction of prompts with at least one successful rollout, necessary indicators for GRPO and SDPO respectively. On the training prompt path both are 32 %. These measurements do not guarantee successful post-training, but identify whether the base checkpoint supplies usable reward variation and successful siblings. Full protocol and prompt-format controls are in Appendix C.

4.5 Compute

Each arm trains for 150 steps on $2 \times \text{H200}$ GPUs under a verl-based [Sheng et al., 2025] trainer. Prover calls execute under bounded time and memory to prevent reward-time resource failures (Appendix D).

5 Results

The measurements below use a scorer we rebuilt and a solver front end we repaired, so we state the instrument’s credentials first. On the reference programs themselves — generator held fixed, so any deviation is ours — our scorer reproduces the published Prover9 row on FOLIO dev exactly: 79.90 % executable against a reference 79.9 %, and 80.37 % accuracy-on-executable against 80.4 %. Applied to untrained open models it locates the bottleneck this paper targets: Qwen2.5-7B answers 62.0 % of the problems it can execute but only 36.8 % overall. Appendix C documents that reproduction and three defects we found in the reference implementation — a reward that depends on PYTHONHASHSEED, an unbounded reward-time resource path, and a nondeterministic evaluation — each of which corrupts a post-training comparison silently rather than loudly.

5.1 Main comparison

Post-training works; the warm start is not the ceiling. The base model reaches 36.27 on ProverQA test with barely half its programs executable (*Exe* 49.27). Thirty steps of GRPO on scalar verifier reward lift that to 66.13, and the full 150-step run to 84.67. Most of the gain is executability: parse errors fall from 476 to 2 and extraction errors from 223 to 24 (Table 3).

Table 3: Failure composition on ProverQA test ($n=1,500$), by arm, as counts. The accuracy column of Table 2 cannot distinguish an arm that stopped reasoning from one that stopped writing parseable programs; this table can. *contra.* is the contradictory-premise degeneracy channel of Section 3.1. The vacuous channel and an internal harness-error channel are tracked and read zero for every arm, so both are omitted. *Exe* is the executable rate; *Exe-Acc* is accuracy restricted to the programs that executed.

	Method	<i>Exe</i>	<i>Exe-Acc</i>	success	extract	parse	clash	solver	<i>contra.</i>
	Base model	49.27	73.61	739	223	476	14	2	46
	GRPO warm start	85.20	77.62	1278	65	34	9	3	111
A-GRPO	GRPO	92.40	91.63	1386	24	2	1	0	87
B-DEMO	SDPO, demo slot	90.40	93.51	1356	56	10	2	0	76
D-SLOT	SDPO, verified program	92.27	92.34	1384	61	12	2	1	40
C-DIAG	SDPO, structured diagnostics	81.07	82.81	1216	105	24	81	6	68
E-CONST	SDPO, constant string	9.47	92.25	142	123	1216	1	6	12
F-MIX	SDPO+GRPO, $\lambda=0.5$	94.53	89.63	1418	12	13	1	1	55

Every contrast below is between SDPO arms at identical mini-batch 1 optimization; GRPO appears as a scalar-reward reference, not as a controlled algorithmic contrast (Section 4.2).

Placement does not matter. B-DEMO and D-SLOT differ in one respect: whether the byte-identical verified program continues the few-shot pattern or appears in an instruction-shaped block after the pattern’s terminator. Same algorithm, optimizer, budget and mini-batch. They score 84.53 and 85.20 — 0.67 points apart — and the per-difficulty and per-label columns agree. We predicted a large effect, from a measurement taken before any arm ran: wrapping the *student’s* prompt in a chat template collapses executability from 12.2 % to 0.9 % (Section 3.2). That sensitivity is real and does not transfer to the teacher’s channel. This is the paper’s central null.

Content matters, but only downward. Holding the slot fixed and varying the payload produces the largest effects in the study, all of them harmful. A verified program scores 85.20, structured diagnostics 67.13 (−18.07), and an information-free constant string 8.73 (−58.40 further). The ordering is the informative part: E-CONST carries no information at all and is by far the worst, so the deficit is a property of what the payload says, not of the slot it occupies. Had E-CONST landed level with D-SLOT, the interface would have been the culprit.

The constant-string collapse is syntactic, not logical. This is the sharpest result in the paper and the easiest to misread. Table 3 shows E-CONST failing to parse on 1,216 of 1,500 problems — 81 % of the test set, against 10 for B-DEMO. But on the 142 programs it does emit, it is 92.25% accurate, level with D-SLOT’s 92.34 and above A-GRPO’s 91.63. The model did not stop reasoning. It stopped writing programs a parser accepts.

Diagnostics teach the defect they describe. C-DIAG’s failure has its own signature. Symbol clashes rise from 2 in B-DEMO and D-SLOT to 81, and extraction errors from 56 to 105, while contradictory-premise rates do not rise. Training on the solver’s complaints about malformed symbols produced a policy that emits malformed symbols. The teacher-quality gate of Section 3.4 refused this channel before any gradient step was taken.

A policy-gradient partner changes nothing. The distillation objective is masked to prompts producing a verified sibling, so groups without one contribute no gradient — 17 % of batches for B-DEMO, 61.6 % for C-DIAG. F-MIX restores a policy-gradient term for those groups and scores 84.73 against B-DEMO’s 84.53. The discarded gradient is not on its own an explanation for the payload ordering.

Out-of-distribution transfer is flat and we do not over-read it. On FOLIO dev ($n=204$) the post-trained arms sit between 40.20 and 50.00, with the GRPO warm start at 48.53 and full GRPO at 46.08. No pair separates beyond evaluation noise at this sample size. There is one readable trade: D-SLOT produces more executable programs OOD (75.49 vs. 65.20) but is less accurate on them (62.34 vs. 76.69). E-CONST is the exception — it scores 32.84, exactly the base model’s number, and

its 81 % parse-failure rate does not reproduce on FOLIO (44 %), so the collapse appears tied to the prompt distribution it trained on rather than to the weights in general.

5.2 What the base model can and cannot be told

The base model cannot use a privileged context at all. On the fixed 100-prompt set the teacher-quality gate scores every candidate context we tried — including a verified-correct program for the exact problem — at or below the plain condition, and the echo audit confirms the few positive margins come from the payload containing the response rather than from the model reading it. After thirty steps of GRPO, the identical verified-program context becomes usable and the diagnostic context does not. The capacity to consume privileged context is installed by post-training, and installed selectively.

That licenses a methodological point which survives our null: a negative inference-time result is routinely reported as a property of the feedback signal, when it is a joint measurement of the base model and the channel. Our own arms bound the argument — once the capacity *is* installed, the diagnostic signal still does not help.

What we did not run. The pre-registered design included two prompting-only arms: P-PROMPT, the LOGIC-LM recipe with no gradient, and R-REFINE, the same recipe with inference-time refinement driven by the diagnostic layer that feeds C-DIAG. Neither was run; the compute went to the factorial. Our inference-time comparison therefore rests on the base-model row of Table 2 and a published closed-model result [Maheshwari and Sharma, 2026], not a matched open-model refinement arm. This is the clearest gap in the paper.

5.3 Post-training dynamics

Figure 1 is the reason validation runs every ten steps rather than at the endpoints: the divergence it exposes is the post-training analogue of the inference-time pathology reported by Maheshwari and Sharma [2026], and the payload ordering is stable across the run rather than a property of the endpoint.

6 Limitations

One seed per arm Every arm in Table 2 is a single seed; the factorial exhausted the budget. The large payload differences are stable across validation checkpoints and the placement null holds in every slice, but one seed cannot establish run-to-run variance or the ordering within the top cluster (A-GRPO, F-MIX, B-DEMO, D-SLOT), and we make no claim about which is best.

Missing matched controls The upstream GRPO and SDPO configurations use different mini-batch sizes (Section 4.2), so we make no controlled SDPO-versus-GRPO claim; the payload and placement conclusions are internal to the MB-1 SDPO family. The prompting-only arms were also not run, so the inference-time comparison relies on the base-model result and prior work rather than a matched open-model control.

Scope All training uses one model, one prover, one formal language, and solver-generated labels. FOLIO provides only a small directional OOD check ($n=204$), so broader generalization remains open. We also leave symbol clashes unrepaired, so our executability numbers are not comparable to work that sanitizes them. Whether larger policies can exploit privileged context without a warm start is likewise unresolved; the viability and teacher-quality gates provide measurements for testing that question in future settings.

7 Conclusion

A theorem prover that rejects a translation already explains why, and that explanation is richer than the scalar reward post-training normally consumes. Whether spending it helps turns out to depend almost entirely on what it says.

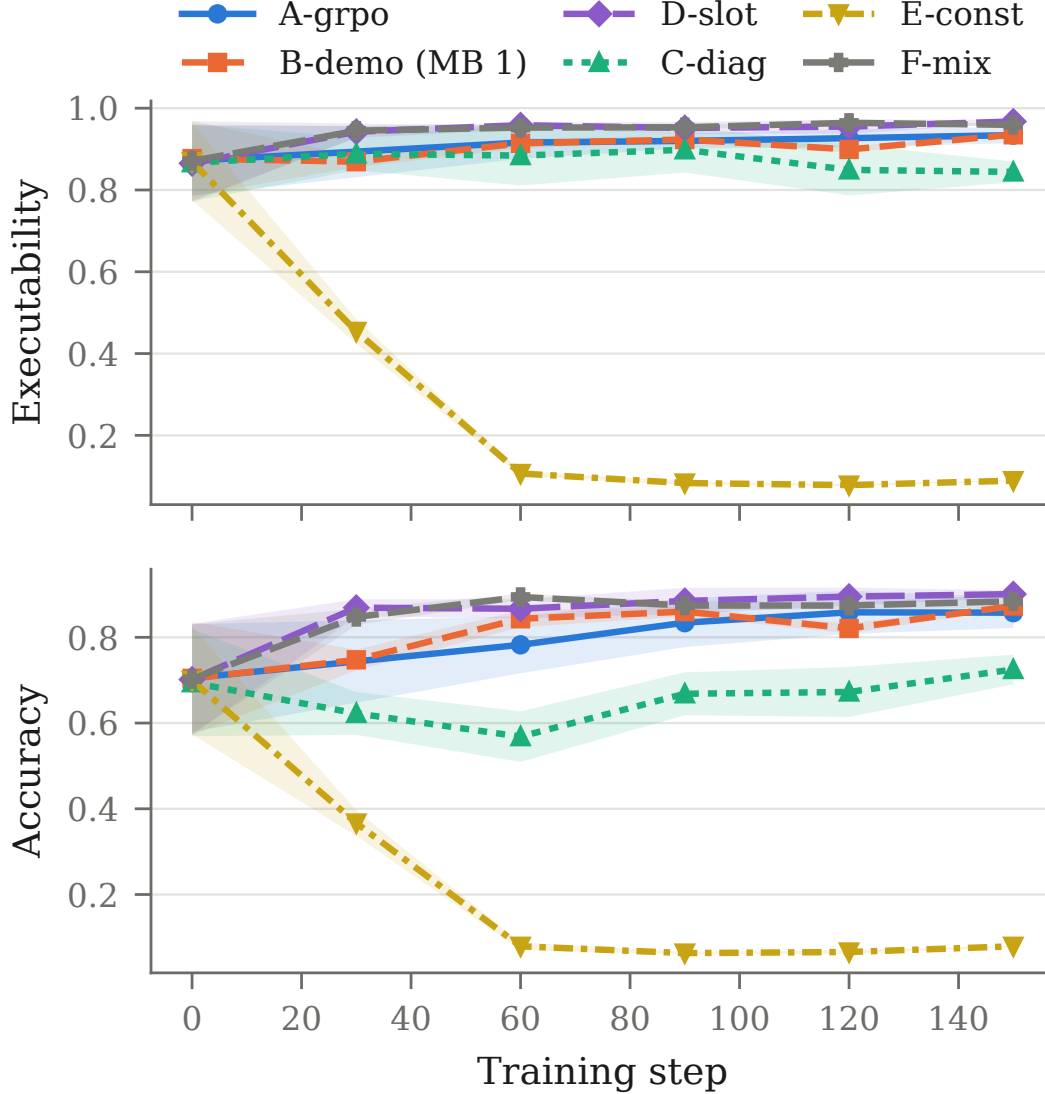


Figure 1: Executability (top) and end-task accuracy (bottom) against training step, single seed per arm, four validation samples per prompt. The pathology to look for is divergence between the two curves: an arm whose executability climbs while accuracy stalls has learned to satisfy the parser rather than to translate.

317 Holding the base model, prover, reward, optimizer and rollout budget fixed and varying only the
318 privileged context given to a self-distillation teacher, placement makes no difference — a byte-
319 identical verified program scores 84.53 in the demonstration slot and 85.20 in the feedback slot, a
320 null we predicted against — while content within one slot spans the whole table: 85.20 for a verified
321 program, 67.13 for the prover’s diagnostics, 8.73 for an information-free constant string. The two
322 harmful payloads fail differently, which is the part that generalizes: one keeps its reasoning and
323 loses its output format, the other starves the distillation objective of targets and learns to produce
324 the malformed symbols its feedback complains about. Both were separable before any gradient
325 step, by a gate whose verdict depends on the model–context pair rather than on the string alone. We
326 additionally document what breaks when a sound solver becomes a reward function: degeneracy
327 channels invisible to accuracy, and determinism and resource failures in a widely forked reference
328 implementation.

329 The question is therefore not whether a symbolic verifier supplies rich supervision. It plainly does. It
330 is whether the current policy can consume it, and that is measurable before the compute is spent.

References

- Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *Proc. International Conference on Learning Representations (ICLR)*, 2024.
- Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in AI safety. *arXiv preprint arXiv:1606.06565*, 2016.
- Luoxin Chen, Yichi Zhou, and Huishuai Zhang. Learning to generate formally verifiable step-by-step logic reasoning via structured formal intermediaries. *arXiv preprint arXiv:2603.29500*, 2026. URL <https://arxiv.org/abs/2603.29500>.
- Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *Proc. International Conference on Learning Representations (ICLR)*, 2024.
- DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Jean-Bastien Grill, Florian Strub, Florent Altché, Corentin Tallec, Pierre H. Richemond, et al. Bootstrap your own latent: A new approach to self-supervised learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- Simeng Han, Hailey Schoelkopf, Yilun Zhao, Zhenting Qi, Martin Riddell, Wenfei Zhou, James Coady, David Peng, Yujie Qiao, Luke Benson, et al. FOLIO: Natural language reasoning with first-order logic. In *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
- Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *Proc. International Conference on Learning Representations (ICLR)*, 2024.
- Jonas Hübotter, Frederike Lübeck, Lejs Deen Behric, Anton Baumann, Marco Bagatella, Daniel Marta, Ido Hakimi, Idan Shenfeld, Thomas Kleine Buening, Carlos Guestrin, and Andreas Krause. Reinforcement learning via self-distillation. *arXiv preprint arXiv:2601.20802*, 2026. URL <https://arxiv.org/abs/2601.20802>.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Devesh Maheshwari and Surbhi Sharma. Beyond self-refinement: Ensembling and chaining for neurosymbolic reasoning. In *ICLR 2026 Workshop on Logical Reasoning of Large Language Models*, 2026. URL <https://openreview.net/forum?id=ISQWwRwD0z>.
- William McCune. Prover9 and Mace4. <https://www.cs.unm.edu/~mccune/prover9/>, 2005–2010.
- Theo X. Olausson, Alex Gu, Benjamin Lipkin, Cedegao E. Zhang, Armando Solar-Lezama, Joshua B. Tenenbaum, and Roger Levy. LINC: A neurosymbolic approach for logical reasoning by combining language models with first-order logic provers. In *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 5153–5176, 2023.
- Theo X. Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. Is self-repair a silver bullet for code generation? In *Proc. International Conference on Learning Representations (ICLR)*, 2024.
- Liangming Pan, Alon Albalak, Xinyi Wang, and William Yang Wang. Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning. In *Findings of the Association for Computational Linguistics: EMNLP*, pages 3806–3824, 2023.

378 Chengwen Qi, Ren Ma, Bowen Li, He Du, Binyuan Hui, Jinwang Wu, Yuanjun Laili, and Conghui
379 He. Large language models meet symbolic provers for logical reasoning evaluation. In *Proc.*
380 *International Conference on Learning Representations (ICLR)*, 2025.

381 Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang,
382 Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of
383 mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

384 Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng,
385 Haibin Lin, and Chuan Wu. HybridFlow: A flexible and efficient RLHF framework. In *Proc.*
386 *European Conference on Computer Systems (EuroSys)*, 2025.

387 Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu
388 Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural*
389 *Information Processing Systems (NeurIPS)*, 2023.

390 Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and
391 characterizing reward hacking. In *Advances in Neural Information Processing Systems (NeurIPS)*,
392 2022.

393 Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2025.

394 Koushik Viswanadha, Deepanway Ghosal, and Somak Aditya. LogicPO: Efficient translation
395 of NL-based logical problems to FOL using LLMs and preference optimization, 2025. URL
396 <https://arxiv.org/abs/2506.18383>.

397 Xi Ye, Qiaochu Chen, Isil Dillig, and Greg Durrett. SatLM: Satisfiability-aided language models
398 using declarative prompting. In *Advances in Neural Information Processing Systems (NeurIPS)*,
399 2023.

400 Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Tiantian Fan, Gaohong
401 Liu, et al. DAPO: An open-source LLM reinforcement learning system at scale. *arXiv preprint*
402 *arXiv:2503.14476*, 2025.

403 A The nine-class failure taxonomy

404 Expanded from Section 3.1. Classes are mutually exclusive and exhaustive; every generation lands in
405 exactly one.

- 406 • `success` — parsed, executed, verdict returned.
- 407 • `extract_error` — no usable program in the generation at all (missing
408 `Premises:/Conclusion:` block, empty output).
- 409 • `parse_error` — syntactically malformed first-order logic.
- 410 • `symbol_clash` — a symbol used at two arities, or as both a predicate and a function.
- 411 • `solver_error` — Prover9 rejected an otherwise well-formed program.
- 412 • `timeout` — the wall-clock or search budget was exhausted.
- 413 • `vacuous_kb` — executable, but the premises do not constrain the conclusion (Section 3.1).
- 414 • `contradictory_kb` — executable, but $K \vdash g$ and $K \vdash \neg g$ (Section 3.1).
- 415 • `internal_harness_error` — *our* defect, not the policy’s. Must read zero; a non-zero
416 value is a bug report, not a finding.

417 **What the reference backend discarded** The information Prover9 produces exists and is thrown
418 away before any caller can see it. In the vendored LOGIC-LM first-order logic backend, the top-level
419 entry point catches every exception with a bare handler and returns **False**; the formula wrapper
420 collapses all failures into an `is_valid` flag; the generated parser’s error hook has its only diagnostic
421 line commented out; and the success path returns the empty string as its message. Rebuilding the
422 layer is what makes ϕ available as an experimental variable at all.

423 **Why internal_harness_error is a separate class** Harness crashes must never be attributed to
 424 the policy, because doing so would train the model away from constructions it was entitled to write.
 425 We report the rate separately and treat a non-zero value as a bug report rather than a finding; repairing
 426 the front end drove it from 15/14/8 items across our three calibration models to zero (Section C.4).

427 **Two failure modes we deliberately leave failing** Symbol clashes and order comparisons ($>$, $<$)
 428 are not repaired. Prover9 has no arithmetic, so admitting order comparisons would produce programs
 429 from which nothing can be derived — an Unknown verdict that maps to “Uncertain” and collects
 430 free reward. Sanitizers that rewrite clashing arities exist in the literature; we do not adopt them. These
 431 are precisely the error modes the post-training signal is supposed to eliminate, and removing them by
 432 hand would delete the phenomenon under study.

433 B Full hyperparameters and compute

434 Values are taken verbatim from the reference SDPO rich-textual-feedback configuration; deviations
 435 are listed explicitly.

Table 4: Post-training configuration. “Shared” rows are identical across all trained arms; SDPO-only rows apply to every SDPO variant (B-DEMO, D-SLOT, C-DIAG, E-CONST, F-MIX), which differ only in the privileged context.

Scope	Parameter	Value
Shared	base model	Qwen2.5-7B-Instruct
Shared	train batch size	32
Shared	rollouts per prompt n	8
Shared	learning rate	1×10^{-6} , no warmup
Shared	importance correction	token-level rollout
Shared	validation samples	4 per prompt
Shared	validation cadence	every 10 steps
Shared	seeds	1
SDPO	α	1.0
SDPO	teacher update rate	0.01
SDPO	top- k	20
SDPO	success threshold τ	1.0
SDPO	re-prompt on self-success	no
SDPO	re-prompt token budget	10240
Confound	mini-batch size	GRPO 8 vs. SDPO 1 (inherited; see §4.2)
Compute	hardware	2×H200 per job
Compute	trainer	verl [Sheng et al., 2025]
Compute	training steps	150

436 **Why the reward’s top tier equals the success threshold** The top tier of Equation 1 is set exactly
 437 at SDPO’s success threshold $\tau = 1.0$, which is not cosmetic: the self-teacher distills only from
 438 trajectories scoring at or above that threshold, so a shaped reward topping out below it would silently
 439 reduce SDPO to GRPO.

440 **F-MIX: normalizing the two objectives** F-MIX optimizes $\mathcal{L} = \lambda \mathcal{L}_{\text{PG}} + (1 - \lambda) \mathcal{L}_{\text{distill}}$ at $\lambda=0.5$.
 441 The two terms carry different normalizations — the distillation loss is normalized over the mini-batch,
 442 the policy-gradient loss per micro-batch — so each is scaled before weighting; blending the raw
 443 values would make the effective λ depend on batch token counts.

444 **The two teacher-quality gate controls** Both controls in Section 3.4 were necessary in practice.
 445 The *foreign-payload control* measures g a second time with a payload of identical shape drawn from
 446 a *different* problem and reports the corrected quantity $g - g_{\text{foreign}}$. Without that subtraction a channel
 447 is credited for the generic benefit of one more well-formed program in context, whether or not it
 448 answers the question asked; in our measurements the correction removes roughly half of the raw
 449 effect. The *echo audit* computes the n -gram coverage of each scored response by its own context

and rejects the measurement when coverage exceeds a threshold. This is not hypothetical: an early version of our sibling channel embedded the very response it was scoring, giving coverage 1.0 and a spuriously large g .

Bounding the prover inside the training loop Each program is executed in an isolated child process under an address-space limit and a monotonic wall-clock deadline enforced by process-group termination, with an explicit memory cap passed to Prover9 and a bound on parent-side output buffering. The rationale is in Section D.2.

B.1 Existing assets and licenses

Table 5: Third-party assets used in this work, with the license of each released artifact as stated by its upstream model card or repository. ProverQA declares no license; its dataset card states that the data is intended for academic and research use and that its sources comply with the licenses of the corpora they derive from. We use it within that stated scope.

Asset	Use	License / terms
Qwen2.5-7B-Instruct	base policy, all trained arms	Apache-2.0
Qwen2.5-7B (base)	untrained baseline (Table 8)	Apache-2.0
Qwen2.5-3B (base)	untrained baseline (Table 8)	Qwen Research License
ProverQA	training and in-distribution test data	none declared; research use
FOLIO	out-of-distribution evaluation	CC-BY-SA-4.0
Prover9/Mace4 (LADR)	theorem prover, reward and evaluation	GPL-2.0
verl (HybridFlow)	post-training framework	Apache-2.0

C Validating the measurement apparatus

Every post-training number in Section 5.1 is produced by a scorer we rebuilt, on a pipeline whose reference implementation we found to be nondeterministic. This appendix establishes that the instrument is trustworthy before the measurements taken with it are believed. Section C.3 shows the scorer reproduces the published reference row exactly; Section C.4 calibrates it on untrained open models and quantifies the translation bottleneck the paper targets; Sections C.5–C.7 document three measurement defects in the reference implementation, each of which silently corrupts a post-training comparison.

C.1 ProverQA label priors

Table 6: ProverQA label distribution. “Uncertain” is 14.9 % of training and 35.9 % of test, so overall accuracy is not comparable across arms without the per-label breakdown. The label prior also shifts across difficulty (A is 36.0 % of easy but 28.6 % of hard). The training file carries no difficulty annotations, so the training mixture cannot be stratified by difficulty. We drop ProofWriter, which the reference pipeline routes to a rule-based backend rather than to Prover9 and which therefore does not exercise the component under study.

Split	n	A (True)	B (False)	C (Uncertain)
train	4,966	2,122 (42.7 %)	2,102 (42.3 %)	742 (14.9 %)
test	1,500	470 (31.3 %)	491 (32.7 %)	539 (35.9 %)

A policy that becomes even slightly better or worse at “Uncertain” moves the headline number through the test prior alone, independently of any reasoning gain, which is why per-label accuracy and its macro-average appear in every table.

C.2 The viability gate: protocol and prompt-format controls

Sampling $n=8$ at temperature 1.0 over 100 prompts costs minutes rather than GPU-days. On the training prompt path the base model gives 32 % of groups with non-zero reward variance, 32 % of prompts with at least one success, and 11.0 % executability.

Table 7: Reproduction of the reference Prover9 row on FOLIO dev ($n=204$, GPT-4 programs, no self-refinement). Tolerance is $\pm 1-2$ items; neither Prover9 nor the parsing stack is version-pinned to the original.

Metric	Reference	Ours	Δ
Executable rate	79.9 %	163/204 = 79.90 %	+0.00
Accuracy on executable	80.4 %	131/163 = 80.37 %	-0.03
Accuracy (non-exec. wrong)	64.24 %	131/204 = 64.22 %	-0.02

Table 8: Untrained baselines on FOLIO dev ($n=204$), zero-shot, greedy decoding, single sample, non-executable scored as incorrect. The gap between accuracy and accuracy-on-executable is the translation bottleneck this paper targets. Dashes mark quantities that are undefined rather than unmeasured: the direct and chain-of-thought modes answer in natural language, so no program exists to execute.

Model	Mode	Exe	Acc.	Acc. on exec.
<i>Natural-language answering — no program to execute</i>				
Qwen2.5-7B-Instruct	direct	—	56.86 %	—
Qwen2.5-7B-Instruct	chain-of-thought	—	68.14 %	—
<i>Neurosymbolic</i>				
Qwen2.5-7B (base)	LOGIC-LM	59.3 %	36.8 %	62.0 %
Qwen2.5-3B (base)	LOGIC-LM	55.9 %	29.9 %	53.5 %
Qwen2.5-7B-Instruct	LOGIC-LM	52.0 %	34.8 %	67.0 %

The measurement is highly sensitive to prompt format, which is the reason Section 3.2 treats placement as a live hypothesis. On a separate 100-prompt sample the same base model gives 33 % / 33 % and 12.25 % executability through the completion-style LOGIC-LM prompt, against 2 % / 2 % and 0.875 % through the model’s chat template — a checkpoint that reads as post-trainable under one presentation and as hopeless under another, with the task, the reward and the available information held identical.

C.3 Reproducing the reference pipeline

Table 7 shows an exact reproduction. Two details are worth recording because they cost us time and are not documented upstream.

First, the reproducible row is the one *without* the chain-of-thought fallback. The frequently quoted higher figures correspond either to round-3 self-refinement or to the fallback-augmented protocol, and neither is reachable from the released artifacts.

Second, **aggregate executable rate is not a valid regression test for this pipeline**. Identical code returns 161 or 163 depending on machine load, because the wall-clock deadline covers *parsing* and three specific items have chart-parser costs that race it. A ± 2 wobble is the same magnitude as the effects a parser change produces, and it nearly caused us to revert a clean forty-item improvement. We therefore gate on a *per-item* manifest, comparing each instance’s status against a stored reference and treating those three items as a labelled known-flaky set. Under the repaired front end the status split is 163 success / 33 parse_error / 8 exec_error, against 163/29/12 before — the aggregate is unchanged while four items are reclassified, which is exactly the kind of movement an aggregate metric cannot see.

C.4 What pre-training left in place: untrained baselines

Table 8 states the problem quantitatively. For Qwen2.5-7B base, the pipeline answers 62.0 % of the problems it can execute but only 36.8 % overall; the 25-point difference is entirely non-executable programs. A model that reasons acceptably is being destroyed by a formalization step. The natural-language rows make that concrete: the same instruct model answers 68.14 % of these problems with chain-of-thought and 56.86 % answering directly, against 34.8 % through the pipeline. Read through this paper’s framing, the split is diagnostic: the reasoning competence is present in the base model, and the *formalization* competence is what pre-training did not install. That is the gap the

Table 9: Failure composition of the untrained baselines (FOLIO dev, $n=204$, item counts). `internal_harness_error` is zero for all three after the front-end repairs of Section 3.1; it read 15/14/8 before.

Model	success	parse	clash	extract	solver	timeout
Qwen2.5-7B base	121	49	26	5	3	0
Qwen2.5-3B base	114	41	38	10	1	0
Qwen2.5-7B-Instruct	106	75	11	9	2	1

post-training signal has to close, and it is why executability and accuracy must be read together rather than separately.

The corresponding failure composition (Table 9) shows the gap is not a single defect. Parse errors dominate, but symbol clashes — a predicate used at two arities, or as both predicate and function — account for 26, 38, and 11 items across the three models, and these are semantic errors, not typos: exactly the class of constraint that a prompt can name but not instill.

Agreement with published reference points Our executable rates match and slightly exceed the two published open-model figures we could locate for this pipeline (0.593 vs. 0.55 and 0.559 vs. 0.52). Our accuracy-on-executable does not agree, by a factor of three to four — and should not. Both published values sit *below* the 33 % chance floor of a three-way classification task, and that table is internally inconsistent in ways we document in Section C.7. We cite those numbers as unverifiable reference points, not as targets.

C.5 The reward function was nondeterministic

While calibrating the pipeline we found a defect that we believe affects any reinforcement-learning use of this widely-forked solver, and that would have made our own results uninterpretable had it survived into training.

The parser’s symbol-resolution step returns Python `set` objects, and the lowering step builds a regular-expression alternation by joining them in iteration order. Set iteration order for strings depends on the process hash seed. The consequence is stark: **the same program returned success under PYTHONHASHSEED 1, 2, 4 and parse_error under 0, 3, 5**. Under a distributed trainer, where reward computation is spread over worker processes that do not share a hash seed, this makes the reward function a coin flip on which worker happened to evaluate a rollout. The policy would be receiving gradient from noise uncorrelated with anything about the program.

The repair — escaping alternation members and ordering them longest-first, which is also required for correctness when one symbol is a prefix of another — makes the outcome deterministic: 612 generations evaluated in two fresh processes produce zero disagreements.

We report this because the failure is silent. It produces no exception, no warning, and no bias in aggregate statistics: an evaluation run averages over the noise and looks fine. Only a per-item, cross-process comparison exposes it. Any pipeline that uses a symbolic verifier as an RL reward should test the reward function for determinism explicitly, as a first-class check rather than an assumption.

And neither is the evaluation The same protocol applied to the eval, which never had one, separates two failures that look identical in aggregate. Two runs of the same checkpoint under identical configuration on identical hardware agree on *every* score they assign to an identical generation — the scorer is deterministic, so no observed difference between arms is a harness artifact. They nevertheless disagree on roughly a sixth of per-item predictions, because greedy decoding is not reproducible: reduction order in batched matrix multiplication depends on which sequences are co-resident, and a last-bit difference in logits flips an argmax that then propagates through the rest of the generation. Pinning the seed, disabling CUDA graphs and prefix caching, fixing the scheduler batch cap and pinning the GPU model all reduce it and none of them remove it. Aggregate accuracy is far more stable than the per-item predictions underneath it, which is why we report end-of-training accuracy on the full test split rather than reading movements off a validation curve. We state this because a post-training comparison whose effect size is smaller than its decoder noise is not measurable, and the noise floor is almost never reported.

546 C.6 Five solver repairs, each gated per item

547 Cumulatively the front-end repairs move untrained executability from 0.559 to 0.593 (7B base), 0.451
548 to 0.559 (3B base), and 0.397 to 0.520 (7B-Instruct), with **zero** success→failure **regressions**
549 at any step across 612 open-model generations and 204 reference generations. The repairs are: a
550 null-guard on the lowering path whose error check had been gated behind a strict flag; stripping of
551 markdown and enumeration prefixes during program extraction; the determinism fix above; support
552 for function terms (`Mortal(Father(x))`); and support for equality and disequality. Each was
553 accepted only after a per-item gate confirmed no instance moved from success to failure. We stress
554 this protocol because the flakiness described in Section C.3 makes aggregate comparison actively
555 misleading at this scale — the noise floor and the effect size are the same order of magnitude.

556 C.7 On published reference points

557 Our accuracy-on-executable figures exceed the two published open-model values we could locate by
558 a factor of three to four. Rather than treat this as a reproduction failure we note that both published
559 values fall below the 33 % chance floor of a three-way task, that the same table reports a 7B model
560 scoring worse than a 3B model on one benchmark and a 1B model beating a 3B model on executable
561 rate, and that the corresponding baseline implementation was never released — so the column cannot
562 be audited by anyone. We report the comparison for completeness and do not treat those values
563 as targets. The executable-rate column, which we do match, is the one that can be independently
564 checked.

565 D Reward-specification audits

566 A sound prover does not imply a sound reward. This appendix documents two ways the reward
567 specification of Section 3.1 can fail independently of the prover’s correctness: the prover’s resource
568 behaviour is reachable by the policy (Section D.2), and the benchmark’s construction can make a
569 sound degeneracy test unavailable (Section D.3).

570 D.1 The incentive each degeneracy creates

571 Section 3.1 defines the two degeneracies; here is why each is an incentive rather than an accident.
572 Under a correctness reward, a *contradictory* knowledge base is the shortest path to “prove whatever
573 is asked”: an inconsistent K entails every formula, so the prover returns the requested verdict and the
574 reward is maximal on a program that encodes nothing. A *vacuous* knowledge base is the symmetric
575 exploit — premises that constrain nothing yield `Unknown`, which maps to “Uncertain”, worth more
576 than a third of our test set. Both checks cost one additional prover call on programs that already
577 succeeded.

578 This yields a falsifiable secondary hypothesis: *if a reward-maximizing arm’s degeneracy rate rises*
579 *while its accuracy is flat, the arm is hacking the reward*. Because the metric is defined on the same
580 rollouts the reward already scores, it costs nothing to report and is informative regardless of the sign
581 of the main result.

582 D.2 The reward function is an attack surface

583 Running a theorem prover inside a training loop exposes failure modes that do not appear when the
584 same code runs as an offline evaluation script, and we found three.

585 First, **no wall-clock bound exists anywhere in the stack**. The bridge to the prover calls the
586 subprocess without a timeout, and the prover’s own `max_seconds` is measured in *user CPU time*,
587 checked once per generated clause. A descheduled process accrues no user CPU, so under parallel
588 load that limit never fires. The reference implementation survives only by accident: it arms a 60-
589 second alarm for the parser and never cancels it, so the stale alarm lands inside the prover call and
590 is swallowed by a bare exception handler. That mechanism is unusable here, because signal-based
591 alarms raise an exception off the main thread — which is where a distributed trainer computes rewards
592 — leaving no guard and no warning.

593 Second, **no memory bound exists either**. The bridge never emits the prover’s memory-cap directive,
594 and the parent buffers the child’s entire output in memory. We observed 7.4 GB of *parent* resident
595 memory within three seconds on a single FOLIO item.

596 Third, and most relevant to RL, **the parser itself is unbounded**. On one FOLIO item a fourteen-
597 conjunct premise explodes the chart parser — the grammar’s binary-operator rule is ambiguous, so
598 the number of parses grows as the Catalan number of the operator count — consuming roughly 5 GB
599 *before the prover is invoked at all*. No prover-side bound covers this, and no alarm bounds memory.

600 The significance is not that these are bugs but that they are *reachable by the policy*. A model can
601 make the reward function allocate gigabytes by emitting a long conjunction, and such formulas are
602 cheap to produce, so reinforcement learning will find them. Our mitigation is to run each program
603 in an isolated child process under an address-space limit and a monotonic deadline enforced by
604 process-group termination, so that a blowup kills a disposable child rather than the trainer. We
605 consider this a general lesson for the post-training stage: *when a symbolic tool becomes a reward*
606 *function, its resource behaviour becomes part of the policy’s action space*.

607 D.3 A benchmark that makes a sound vacuity test impossible

608 The vacuity check of Section 3.1 is stated as a groundedness test: the conclusion’s predicates must
609 occur in the premises. On FOLIO this is nearly free, costing a single item. On ProverQA it is not,
610 and the reason is structural rather than a matter of threshold.

611 ProverQA *constructs* its Uncertain class by asking about a property the premises never mention.
612 Measured over the full development split, the benchmark’s own reference formalization has a
613 grounded conclusion for 99.8 % of True and 99.8 % of False items, but only 22.8 % of Uncertain
614 ones. The faithful answer and the reward hack are therefore the same shape: both leave the conclusion
615 unconstrained, because that is what makes the label Uncertain in the first place. Grounding the
616 conclusion’s *constants* rather than its predicates does not help either — gold-Uncertain items routinely
617 ask about an entity the premises never name.

618 Scored against reference formalizations, the consequence is stark. The groundedness test zeroes
619 78.9 % of gold-Uncertain reference programs — formalizations the benchmark itself endorses —
620 while blocking the canonical exploit on 100 % of items. There is no setting of the test that is both
621 sound and complete, because the two populations are not separated by the feature it measures.

622 We therefore move the test to the premise side: a program is vacuous when *every* premise is a logical
623 validity, which is what the exploit requires and what a faithful formalization never does (zero of
624 the 15,842 reference premises in the split are validities). This zeroes no reference program in any
625 label class, still blocks the canonical exploit and its excluded-middle variant on 100 % of items, and
626 reduces the FOLIO cost from four executable items to one — and from one correct answer to none.

627 The residual is real and we report it rather than hiding it: an exploit using a single *contingent* but
628 irrelevant premise passes the premise-side test. Ruling it out requires comparing the program against
629 the problem’s own premise list, which the reward function does not receive. We therefore track
630 *ungrounded_conclusion* and the count of non-vacuous premises as per-step metrics, so that a
631 policy drifting toward this exploit is visible as a rising curve rather than paid for silently.

632 The general lesson is uncomfortable and, we think, underappreciated: a verifier can be sound, a
633 reward derived from it can still be unsound, and whether a given soundness repair is available at all
634 depends on how the benchmark was generated. Reward specifications should be audited against the
635 benchmark’s construction procedure, not only against intuition about what a good program looks like.

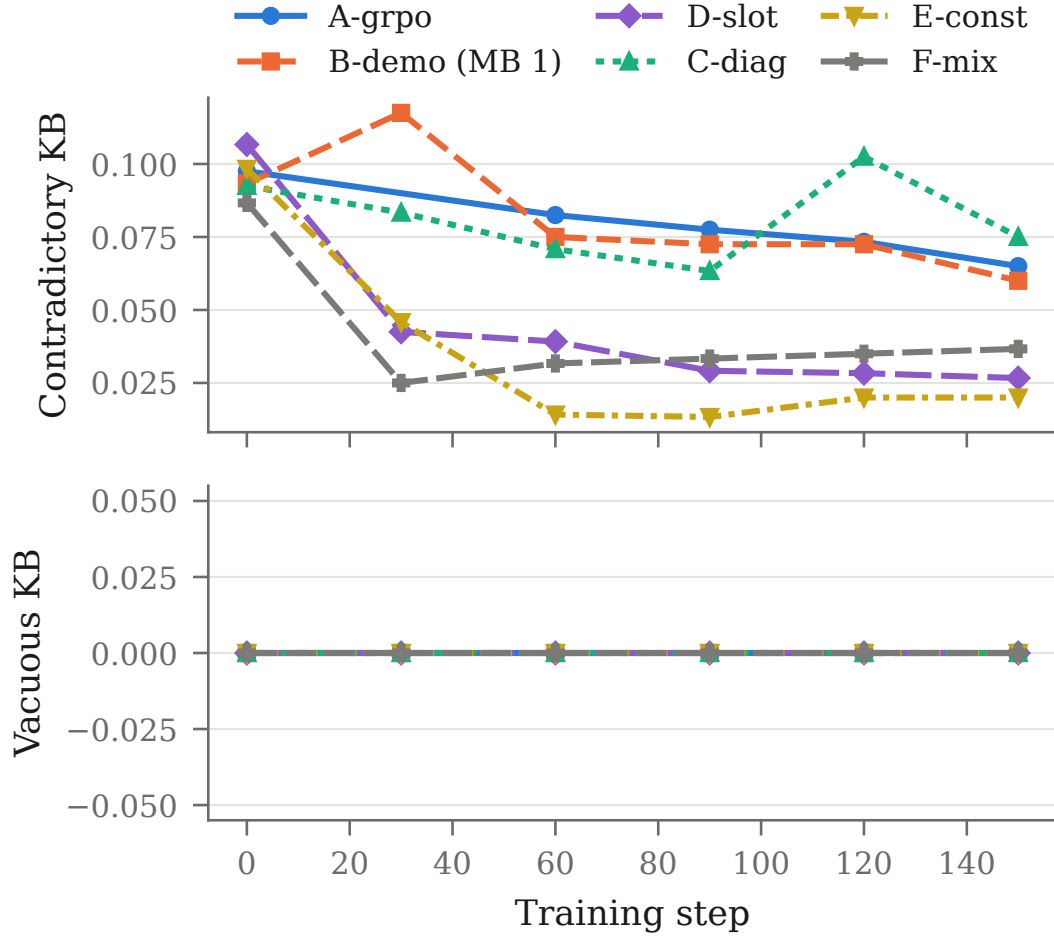


Figure 2: Degeneracy rates during post-training. A rising curve under flat accuracy is reward hacking made measurable (Section 3.1).

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and Section 1 state the payload $\times$ slot factorial, the placement null, and the content effects, each of which is reported in Section 5.1. Scope is stated as conditional on one base model and one prover.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Section 6 is a dedicated Limitations section covering single-seed training, missing matched controls, and scope.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper reports an empirical study and contains no theorems or formal proofs.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Section 3.3 specifies every arm, Appendix B gives the full post-training configuration, and Appendix C documents the evaluation protocol and the per-item regression gate used to make the scorer reproducible.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: The experiments use publicly available benchmarks and open-weight models. The submission venue does not provide an accompanying code-upload mechanism, so code is not included with the anonymous submission. The paper documents the experimental arms, hyperparameters, training configuration, evaluation protocol, and compute requirements needed to reproduce the main experiments. The implementation will be made publicly available with the de-anonymized version of the work.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Section 4 gives datasets, splits and metrics; Appendix B tabulates the optimizer, learning rate, batch and rollout settings, and the SDPO-specific parameters.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: Each arm is a single seed, so we report no error bars and no significance tests. Effect sizes are given in accuracy points, and Section 6 states what a single seed does and does not support, including that we make no claim about the ordering within the top cluster.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.

- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Section 4.5 states that each arm trains for 150 steps on 2×H200 GPUs under a verl-based trainer, with bounded time and memory for prover calls.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The work post-trains an open-weight model on public logical-reasoning benchmarks, involves no human subjects or personal data, and preserves anonymity.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [N/A]

Justification: The work is a controlled methodological study of how privileged supervision is consumed during post-training on formal-logic benchmarks, with no direct path to a deployed application.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper releases no scraped data and no model with elevated misuse risk; all training data is drawn from existing public logical-reasoning benchmarks.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: All existing datasets, models, software packages, and solver components used in this work are cited in the paper. Their respective licenses and terms of use are documented in Appendix B.1, and the experiments comply with those terms.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A]

Justification: The paper does not introduce or release a new dataset, pretrained model, benchmark, or standalone software artifact as part of this submission. The new solver instrumentation and experimental implementation are used to conduct the reported study rather than presented as a released research asset.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper involves no crowdsourcing and no research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

953 Answer: [N/A]

954 Justification: The paper involves no crowdsourcing and no research with human subjects.

955 Guidelines:

- 956 • The answer [N/A] means that the paper does not involve crowdsourcing nor research
- 957 with human subjects.
- 958 • Depending on the country in which research is conducted, IRB approval (or equivalent)
- 959 may be required for any human subjects research. If you obtained IRB approval, you
- 960 should clearly state this in the paper.
- 961 • We recognize that the procedures for this may vary significantly between institutions
- 962 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
- 963 guidelines for their institution.
- 964 • For initial submissions, do not include any information that would break anonymity (if
- 965 applicable), such as the institution conducting the review.

966 **16. Declaration of LLM usage**

967 Question: Does the paper describe the usage of LLMs if it is an important, original, or

968 non-standard component of the core methods in this research? Note that if the LLM is used

969 only for writing, editing, or formatting purposes and does *not* impact the core methodology,

970 scientific rigor, or originality of the research, declaration is not required.

971 Answer: [Yes]

972 Justification: LLM assistance in preparing this manuscript was limited to rephrasing, editing

973 and grammar correction of the prose. They were not used to generate experimental results,

974 conduct analyses, or make scientific conclusions.

975 Guidelines:

- 976 • The answer [N/A] means that the core method development in this research does not
- 977 involve LLMs as any important, original, or non-standard components.
- 978 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
- 979 be described.